Unit-5
Pointers

Pointers

- A pointer is a derived data type that can store the address of other variables or a memory location.
- These variables could be of any type- char, int, function, array, structure, or other pointers.
- Pointers are used to access the memory of the variables and then manipulate their addresses in a program
- Syntax of C pointers:

  The syntax of pointers is similar to the variable declaration in C, but

  we use the (*) dereferencing operator in the pointer declaration.

datatype * ptr;

Where, ptr is the name of the pointer

datatype is the type of data it is pointing to

Pointer Declaration

- To declare a pointer, (*) dereference operator is used before its name.

- Example

  int *ptr;

  The pointer declared here will point to some random memory address as it is not initialized. Such pointers are called wild pointers.

Pointer Initialization

- Pointer initialization means assigning some initial value to the pointer variable.

- (&) addressof operator is used to get the memory address of a variable and then store it in the pointer variable.

- Example

  int var = 10;

int * ptr;

ptr = &var;

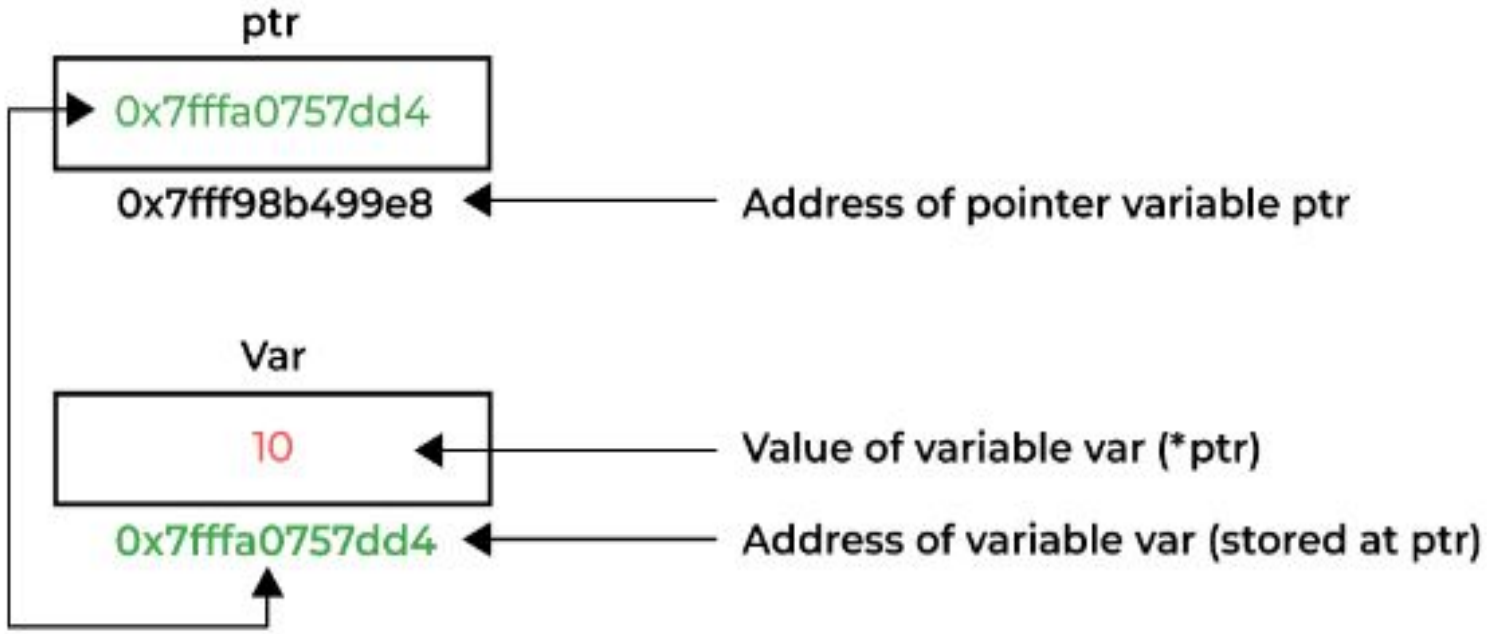
We can also declare and initialize the pointer in a single step. This method is called pointer definition as the pointer is declared and initialized at the same time.

Example

int *ptr = &var;

Pointer Dereferencing

- Dereferencing a pointer is the process of accessing the value stored in the memory address specified in the pointer. The same (*) dereferencing operator is used that is used in the pointer declaration



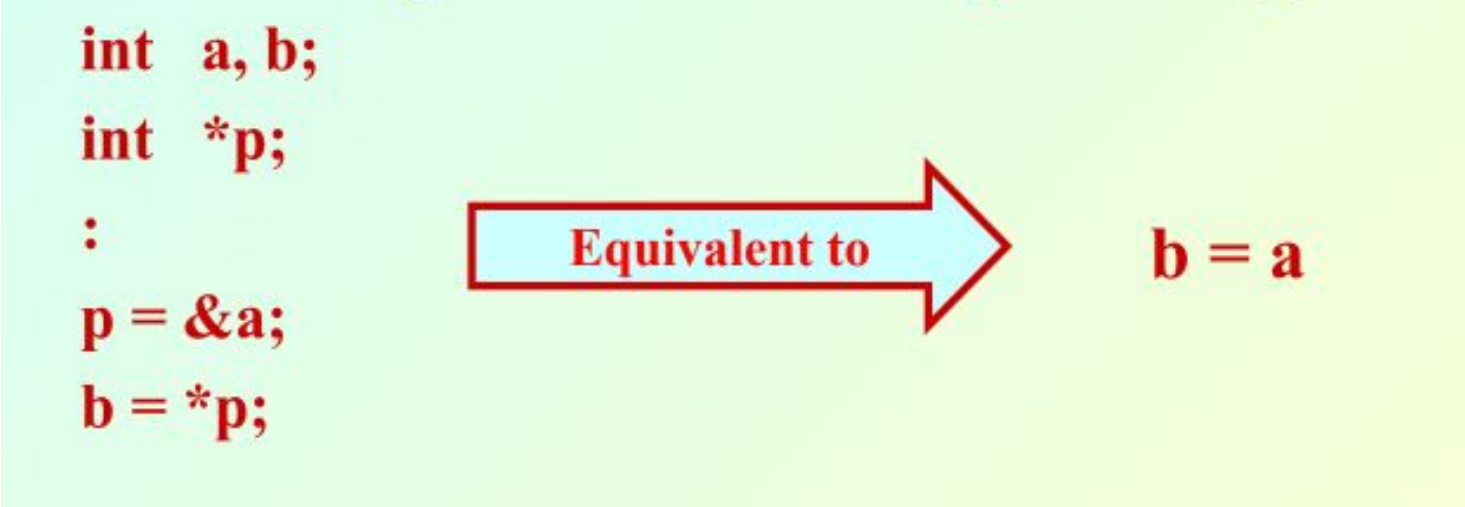
C Pointer Example

```c
#include <stdio.h>

int main()
{
    int var = 10;   // declare pointer variable
    int* ptr; // data type of ptr and var must be same
    ptr = &var;    // assign the address of a variable to a pointer
    printf("Value at ptr = %p \n", ptr);
    printf("Value at var = %d \n", var);
    printf("Value at *ptr = %d \n", *ptr);
    return 0;
}
```

```
C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe
Value at ptr = 000000000062FE24
Value at var = 10
Value at *ptr = 10

--------------------------------
Process exited with return value 0
Press any key to continue . . .
```

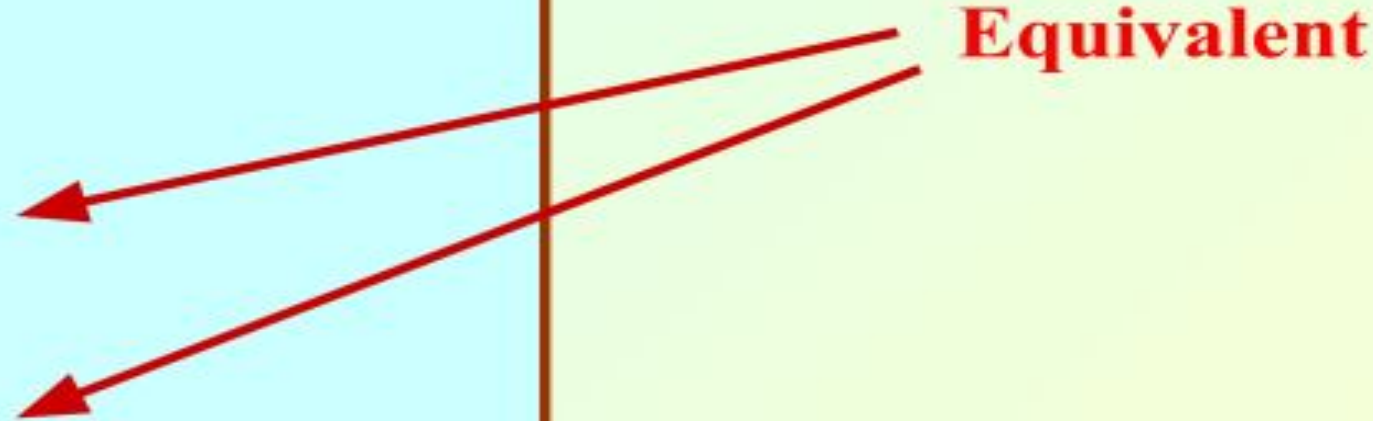
Accessing a Variable Through its Pointer

- Once a pointer has been assigned the address of a variable, the value of the variable can be accessed using the indirection operator (*).



Example 1

```
#include <stdio.h>
main()
{
   int  a, b;
   int  c = 5;
   int  *p;

   a = 4 * (c + 5) ;

   p = &c;
   b = 4 * (*p + 5) ;
   printf ("a=%d  b=%d \n",  a, b) ;
}
```



```c
#include <stdio.h>
main()
{
int x, y;
int *ptr;
x = 10 ;
ptr = &x ;
y = *ptr ;
printf ("%d is stored in location %u \n", x, &x) ;
printf ("%d is stored in location %u \n", *&x, &x) ;
printf ("%d is stored in location %u \n", *ptr, ptr) ;
printf ("%d is stored in location %u \n", y, &*ptr) ;
printf ("%u is stored in location %u \n", ptr, &ptr) ;
printf ("%d is stored in location %u \n", y, &y) ;
*ptr = 25;
printf ("\nNow x = %d \n", x);
}
```

***&x⇔x**

ptr=&x;
&x⇔&*ptr

C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe

```
10 is stored in location 6487596
10 is stored in location 6487596
10 is stored in location 6487596
10 is stored in location 6487596
6487596 is stored in location 6487584
10 is stored in location 6487592

Now x = 25

--------------------------------
Process exited with return value 13
Press any key to continue . . .
```

Types of Pointers in C

- **Integer Pointers**: These are the pointers that point to the integer values.

  int *ptr;

- **Array Pointer:** Pointers and Array are closely related to each other. Even the array name is the pointer to its first element. They are also known as Pointer to Arrays. We can create a pointer to an array using the given syntax.

  Syntax

  char *ptr = &array_name;

Array Pointer

```c
#include <stdio.h>
int main()
{
    int v[3] = { 10, 100, 200 }; // Declare an array
    int* ptr;  // Declare pointer variable
    int i;
    ptr = v;   // Assign the address of v[0] to ptr
    for (i = 0; i < 3; i++) {
        printf("Value of *ptr = %d\n", *ptr); // print value at address which is stored in ptr
        printf("Value of ptr = %p\n\n", ptr);   // print value of ptr
        ptr++; // Increment pointer ptr by 1
    }
    return 0;
}
```

```
C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe
Value of *ptr = 10
Value of ptr = 000000000062FE10

Value of *ptr = 100
Value of ptr = 000000000062FE14

Value of *ptr = 200
Value of ptr = 000000000062FE18
```

Array Pointer: Function to find average

```c
#include <stdio.h>
float avg (int array[ ],int size)   //int * array
{
int *p, i , sum = 0;
p = array ;
for (i=0; i<size; i++)
sum = sum + *(p+i);   //*(p+i)=p[i]
return ((float) sum / size);
}
main()
{
int x[100], i, n ;
printf ("Enter no of elements");
scanf ("%d", &n) ;
for (i=0; i<n; i++)
scanf ("%d", &x[i]) ;
printf ("\nAverage is %f",avg (x, n));
}
```

```
C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe
Enter no of elements3
2 4 6

Average is 4.000000
--------------------------------
Process exited with return value 20
Press any key to continue . . .
```

Structure Pointer

- The pointer pointing to the structure type is called Structure Pointer or Pointer to Structure. It can be declared in the same way as we declare the other primitive data types.
- The name of an array stands for the address of its zero-th element.
  – Also true for the names of arrays of structure variables.
- Consider the declaration:

```
struct stud {
        int  roll;
        char dept_code[25];
        float cgpa;
      } class[100], *ptr ;
```

- The name class represents the address of the zero-th element of the structure array.
  - ptr is a pointer to data objects of the type struct stud.
- The assignment

  ptr = class ;

will assign the address of class[0] to ptr.

- When the pointer ptr is incremented by one (ptr++) :
  - The value of ptr is actually increased by sizeof(stud).
  - It is made to point to the next record.

- Once ptr points to a structure variable, the members can be accessed as:

```
ptr -> roll ;
ptr -> dept_code ;
ptr -> cgpa ;
```

– The symbol "–>" is called the arrow operator.

Dynamic Memory Allocation

- Many a time we face situations where data is dynamic in nature.
  - Amount of data cannot be predicted beforehand.
  - Number of data item keeps changing during program execution.

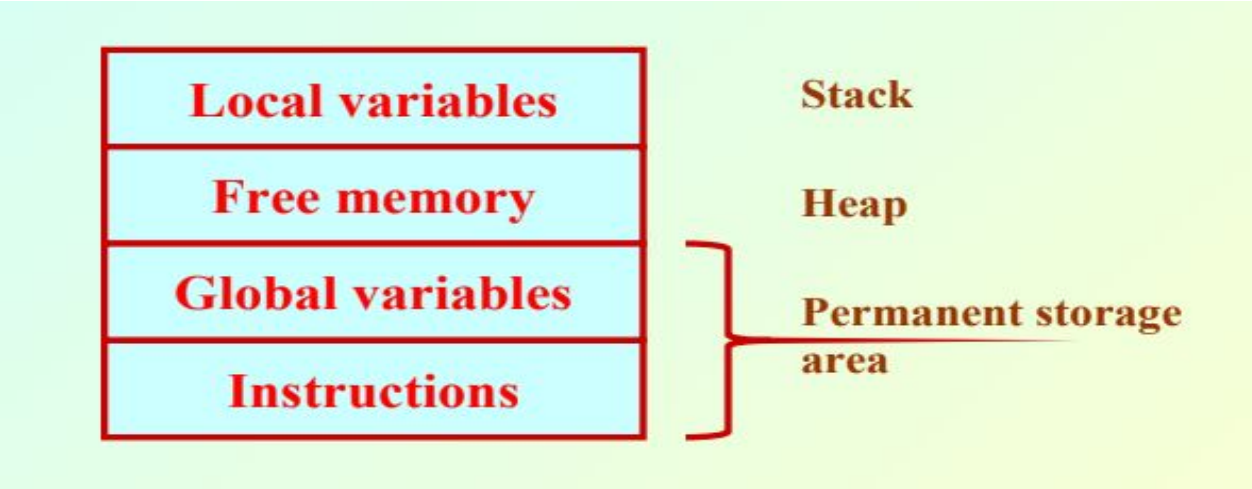
- Such situations can be handled more easily and effectively using dynamic memory management techniques.

Dynamic Memory Allocation

- C language requires the number of elements in an array to be specified at compile time.
  - Often leads to wastage or memory space or program failure.

- Dynamic Memory Allocation
  - Memory space required can be specified at the time of execution.
  - C supports allocating and freeing memory dynamically using library routines.

Memory allocation process in C



- The program instructions and the global variables are stored in a region known as permanent storage area.
- The local variables are stored in another area called stack.
- The memory space between these two areas is available for dynamic allocation during execution of the program.

– This free region is called the heap.
– The size of the heap keeps changing

Memory allocation fuctions

- malloc
  – Allocates requested number of bytes and returns a pointer to the first byte of the allocated space.
- calloc
  – Allocates space for an array of elements, initializes them to zero and then returns a pointer to the memory.
- free
  - Frees previously allocated space.
- realloc
  – Modifies the size of previously allocated space.

Allocating a Block of Memory

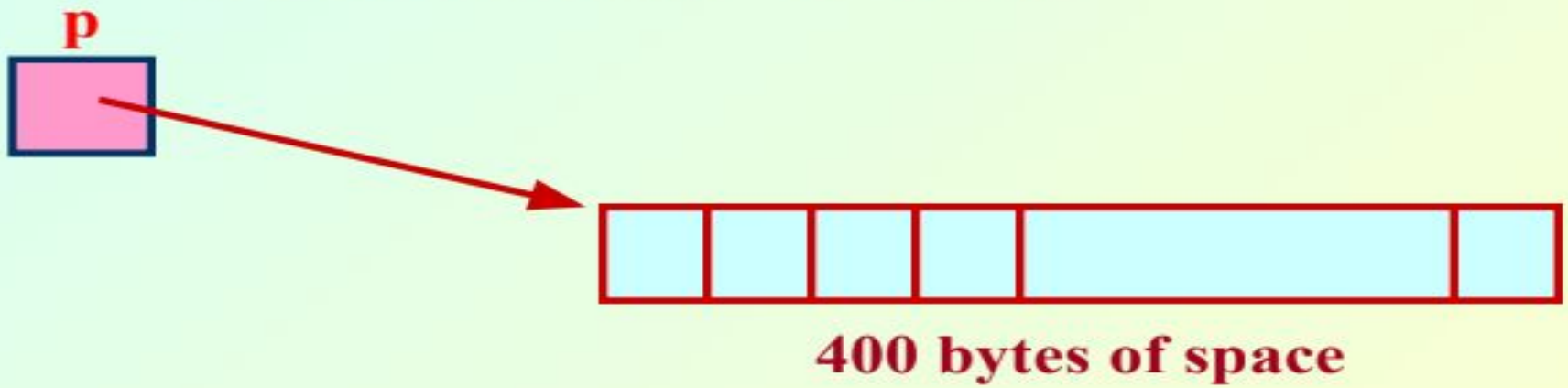
- A block of memory can be allocated using the function malloc.
  – Reserves a block of memory of specified size and returns a pointer of type void.
  – The return pointer can be assigned to any pointer type.
- General format:

```
ptr = (type *) malloc (byte_size) ;
```

- **Examples**

```
p = (int *) malloc (100 * sizeof (int)) ;
```

  - A memory space equivalent to "100 times the size of an int" bytes is reserved.
  - The address of the first byte of the allocated memory is assigned to the pointer p of type int.




```
cptr = (char *) malloc (20) ;
```

```
sptr = (struct stud *) malloc (10 *
                               sizeof (struct stud));
```

Points to note

- malloc always allocates a block of contiguous bytes.
  – The allocation can fail if sufficient contiguous memory space is not available.
  – If it fails, malloc returns NULL.

Example

```c
#include <stdio.h>
main()
{
int i,N;
float *height;
float sum=0,avg;
printf("Input the number of students. \n");
scanf("%d",&N);
height=(float *) malloc(N * sizeof(float));
printf("Input heights for %d students \n",N);
for(i=0;i<N;i++)
  scanf("%f",&height[i]);
for(i=0;i<N;i++)
  sum+=height[i];
avg=sum/(float) N;
printf("Average height= %f \n",avg);
}
```

```
C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe
Input the number of students.
3
Input heights for 3 students
5 6 5
Average height= 5.333333

--------------------------------
Process exited with return value 26
Press any key to continue . . .
```

Releasing the Used Space

- When we no longer need the data stored in a block of memory, we may release the block for future use.
- How?
  – By using the free function.
- General format:

  free (ptr) ;

  where ptr is a pointer to a memory block which has been already created using malloc.

Altering the Size of a Block

- Sometimes we need to alter the size of some previously allocated memory block.
  - More memory needed.
  - Memory allocated is larger than necessary.
- How?
  - By using the realloc function.
- If the original allocation is done by the statement

```
ptr = malloc (size) ;
```

then reallocation of space may be done as

```
ptr = realloc (ptr, newsize) ;
```

Altering the Size of a Block

-The new memory block may or may not begin at the same place as the old one.

- If it does not find space, it will create it in an entirely different region and move the contents of the old block into the new block.

-The function guarantees that the old data remains intact.

-If it is unable to allocate, it returns NULL and frees the original block.